Mastermind

Assignment 1
Semester 1, 2025
CSSE7030

Due date: Friday 11 April 2025, 15:00 GMT+10

1 Introduction

In this assignment, you will implement a modified text-based version of Mastermind, a code-breaking game developed in the 1970's. In this game the player must use a limited number of guesses to determine a secret key. The key is usually represented by an ordered sequence of colors, but in our text based version, we will use numbers instead. Each time the player guesses a key, they are told how many times they guessed a number that appears in the secret key in the same location as their guess, and how many times they guessed a number that appears in the secret key in a different location to their guess. This website presents a version of Mastermind if you wish to try this game for yourself. You will also be implementing an additional hint system allowing the player to request some information about the secret key in the event they are stuck.
A game of mastermind ends when either:

1. The player *wins* by successfully guessing the secret key.
2. The player *loses* because they did not successfully guess the secret key on their last guess.

2 Getting Started

Download `a1.zip` from Blackboard — this archive contains the necessary files to start this assignment. Once extracted, the `a1.zip` archive will provide the following files and folders:

`a1.py`: *This is the only file you will submit* and is where you write your code. *Do not* make changes to any other files.

`support.py`: *Do not modify or submit this file*, it contains pre-defined constants and helper functions to use in your assignment. In addition to these, you are encouraged to create your own constants and helper functions in `a1.py` where possible.

`gameplay/`: This folder contains a number of example outputs generated by playing the game using a fully-functional completed solution to this assignment. The purpose of the files in this folder is to help you understand how the game works, and how output should be formatted.

NOTE: **You are not permitted to add any additional import statements to `a1.py`. Doing so will result in a deduction of up to 100% of your mark.** You must not modify or remove the import statements already provided to you in `a1.py`. Removing or modifying these existing import statements may result in your code not functioning, and in some cases will result in a deduction of up to 100% of your mark.

3 Gameplay

This section provides an overview of gameplay. Where prompts and outputs are not explicitly mentioned in this section, please see Section 4 and the example games in the `gameplay/` folder provided with this assignment.

The game begins with a random key being selected without the players knowledge. This key is five numbers long, with numbers drawn randomly from 1,2,3,4, and 5. It is not necessary for every number to appear in the secret key, and the key may contain duplicate numbers. The player is then presented with an empty board of 10 rows, separated into two groups of 5 columns. The left hand group of 5 columns represent the players guesses, and the righthand set of columns represent the feedback given about these guesses. Above the board is a set of 5 question marks, which will be revealed one by one as the player requests hints.
Until the end of the game, the following steps occur:

1. The current game board state is displayed, along with any hints the player has recieved so far.

2. The user is prompted to enter a command. See Table 1 for the set of valid commands and the actions performed when they are entered. The `gameplay/` folder provided with this assignment presents specific examples for what to do on each command.

3. If the player's command is invalid for any reason, the user is shown a message to inform them of why their command was invalid (see Table 2 for all required validity checking and messages for this step), and then the program returns to step 2. If the user provides a valid guess, the program progresses to the next step.

4. The player's guess is added to the left hand side of the board, below their previous guess (or in the top row if there is no previous guess).

5. Feedback on the player's guess is added to the right hand side of the board next to the guess. Feedback is a number of black markers that counts the number of times the player guessed a number that appears in the secret key in the same location as their guess, followed by a number of white markers that count the number of times the player guessed a number that appears in the secret key in a different location to their guess. See Figure 1 for an example of the feedback the player should recieve.

6. If the game has ended (Due to either a win or a loss), the program continues to the next step. Otherwise, the program returns to step 1.

7. When the game has ended, the player is informed of the outcome.

8. The player is prompted as to whether they would like to play again. At this prompt, if they enter either 'y' or 'Y', a new game is created (i.e. another secret key is selected, and an empty board is set up) and the program returns to step 1. If they enter anything other than 'y' or 'Y', the program should terminate gracefully (that is, the program should end without causing any errors or exiting the test suite).

Valid Command	Action to take
"{number},{number},{number},{number},{number}", where each {number} may be "1","2","3", "4" or "5"	Enter a guess corresponding to the given sequence of characters
"h" or "H"	The user is shown a help message, and then prompted for another command
"t" or "T"	If the user has recieved less than 3 hints, and has made at least 3 guesses, they recieve a hint. Otherwise, the user is shown a message explaining why they are not allowed to recieve a hint. If the user recievied a hint, they are presented the first X numbers of the secret key, where X is the number of hints the player has recieved this game.
"q" or "Q"	gameplay does not continue and the program skips to step 7

Table 1: Valid commands and the actions that should be taken. If the command entered by the user does not *exactly* match one of the commands in this table then no action should be taken for step 2, and the program should advance directly to step 3

Issue with user input	Constant in `support.py`
The guess is not exactly 5 comma separated items	`INVALID_FORMAT_MESSAGE`
At least one of the 5 given items is not either "1","2","3", "4" or "5"	`INVALID_NUMBER_MESSAGE`

Table 2: Constants containing the messages to display when invalid user input is entered. Precedence is **top down** (i.e. if there are multiple issues with user input, only display the message for the one which occurs first in this table).

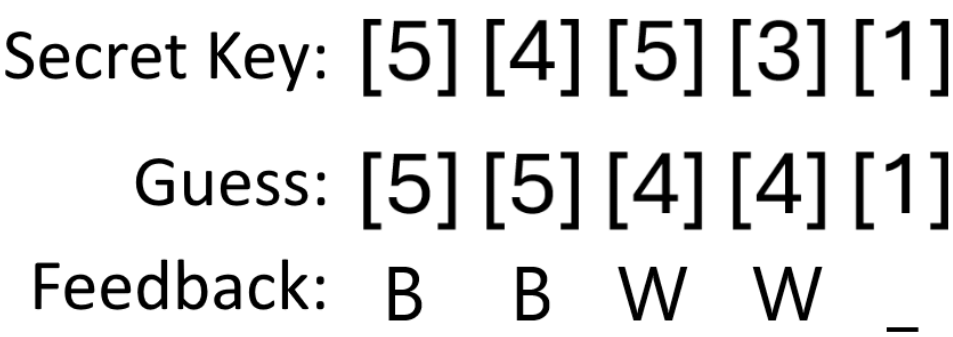

Figure 1: Example of feedback. Here there are two 'Black' markers as the first and last numbers of the guess match the secret key exactly. There are also two 'White' markers as the second and third positions of the guess contain numbers that appear in a different position in the secret key.

4 Implementation

Permitted Techniques:

This assesment has been designed to allow you to practice what you have learnt in this course so far. As such, you ***must only use*** the functions, operators and data types presented to you in lectures up to (and including) **Lecture 4**. Namely, the following techniques are permitted for use in this assignment:

- Functions (`def`,`return`)
- Basic control structures (`for`, `while`, `if/elif/else`, `break`)
- Primitive data types (`int`, `str`, `bool`)
- Variable assignment (`=`)
- Arithmetic (`+`,`-`,`*`,`\`,`\\`, ,`%` etc.)
- Comparison (`==`,`<=`,`>=`,`<`,`>`,`!=` etc.)
- Basic Logic (`not`, `and`, `or` etc.)
- `lists` and `tuples`
- `strings` and `lists` methods
- `range` and `enumerate`
- `input` and `print`

Using any functions, operators and data types that have not been presented to you in lectures up to (and including) **Lecture 4** will result in a deduction of up to 100% of your mark. The following techniques are **not** permitted for use in this assignment:

- Functional programming
- List comprehension
- Lambda functions
- Recursion
- Try-except
- One line `if-else`
- `match`
- Generators

A pinned thread will be maintained on the Ed discussion board with a list of permitted techniques. If you would like clarification on whether you are permitted to use a specific technique, please first check this list. If the technique has not been mentioned, please ask about permission to use the technique in a comment on this pinned thread.

Required Functions

This section outlines the functions you are **required** to implement in your solution (in `a1.py` only). You are awarded marks for the number of tests passed by your functions when they are tested *independently* of one another. Thus an incomplete assignment with *some* working functions may well be awarded more marks than a complete assignment with faulty functions. Your program must operate *exactly* as specified. In particular, your program's output must match *exactly* with the expected output. Your program will be marked automatically so minor differences in output (such as whitespace or casing) *will* cause tests to fail resulting in a *zero mark* for that test.

Each function is accompanied with some examples for usage to help you *start* your own testing. You should also test your functions with other values to ensure they operate according to the descriptions.

The following functions **must** be implemented in `a1.py`. They have been listed in a rough order of increasing difficulty. This does not mean that earlier functions are necessarily worth less marks than later functions. It is *highly recommended* that you do not begin work on a later function until each of the preceding functions can *at least* behave as per the shown examples. You may implement additional functions if you think they will help with your logic or make your code easier to understand.

4.1 `num_hours() -> float`

This function should return the number of hours you estimate to spend (or have spent so far) on the assignment, as a *float*. Ensure this function passes the relevant test on Gradescope *as soon as possible*. The test will only ensure you have created a function with the correct name and number of arguments, which returns a float and does not prompt for input. You will not be marked incorrect for returning the 'wrong' number of hours. The purpose of this function is to let you verify that you understand how to submit to Gradescope as soon as possible, and to allow us to gauge difficulty level of this assignment in order to provide the best possible assistance. You will not be marked differently for spending more or less time on the assignment.

If the Gradescope tests have been released, you must ensure this function passes the relevant test before seeking help regarding Gradescope issues for any of the later functions. See Section 5.3 for instructions on how to submit your assignment to Gradescope.

Example:

```
>>> num_hours()
10.0
```

4.2 `generate_initial_board(board_size: int) -> list[list[str]]`

Returns the initial board state (i.e. an empty board state) with the given number of rows. The board is represented by a list of lists of strings. Each row is represented by a list of strings. The first list in the overarching list represents the top row, and the last list represents the bottom row. The first character in each row list represents the leftmost column, and the last character represents the right most column. The first five characters in each row give the left hand side of the game board (The guess side), and the last five characters in each row give the right hand side of the game board (The feedback side). Note that the guess side of the board and the feedback side of the board use different empty strings.

A precondition to this function is that the given number of rows is greater than 0.

Example:

```
>>> generate_initial_board(2)
[['___', '___', '___', '___', '___', '_', '_', '_', '_', '_'], ['_
__', '___', '___', '___', '___', '_', '_', '_', '_', '_']]
>>> generate_initial_board(3)
[['___', '___', '___', '___', '___', '_', '_', '_', '_', '_'], ['_
__', '___', '___', '___', '___', '_', '_', '_', '_', '_'], ['___',
 '___', '___', '___', '___', '_', '_', '_', '_', '_']]
```

4.3 display_board(board: list[list[str]]) -> None

Prints the game board to the terminal with columns separated by spaces, and the two halves of the game board separated with two sets of spaced pipe characters '`|| ||`', and with rows numbered on the left. The printed output must *exactly* match the format as presented in examples. Note that different system fonts may cause spacing to *appear* different on your machine.

A precondition to this function is that each row of the given board state will contain a number of elements exactly twice the length of the secret key. You may also assume as precondition that the given board state will have less than 100 rows, but you may *not* assume that the given board state will have less than 10 rows. You should not perform any additional validity checking (that is, do not check that the board represents a valid game state).

Example:

```
>>> board1 = generate_initial_board(10)
>>> display_board(board1)
1  ___ ___ ___ ___ ___ || || _ _ _ _ _
2  ___ ___ ___ ___ ___ || || _ _ _ _ _
3  ___ ___ ___ ___ ___ || || _ _ _ _ _
4  ___ ___ ___ ___ ___ || || _ _ _ _ _
5  ___ ___ ___ ___ ___ || || _ _ _ _ _
6  ___ ___ ___ ___ ___ || || _ _ _ _ _
7  ___ ___ ___ ___ ___ || || _ _ _ _ _
8  ___ ___ ___ ___ ___ || || _ _ _ _ _
9  ___ ___ ___ ___ ___ || || _ _ _ _ _
10 ___ ___ ___ ___ ___ || || _ _ _ _ _
         Guesses              Feedback
>>> board2 = [['[5]','[5]','[4]','[4]','[2]','B','W','W','_','_'],
['[5]','[5]','[4]','[4]','[1]','B','B','W','W','_'],['[5]','[4]','
[5]','[3]','[1]','B','B','B','B','B']]
>>> display_board(board2)
1  [5] [5] [4] [4] [2] || || B W W _ _
2  [5] [5] [4] [4] [1] || || B B W W _
3  [5] [4] [5] [3] [1] || || B B B B B
         Guesses              Feedback
>>> board3 = [['C', 'S', 'S', 'E', '7030', 'R', 'O','C','K','S']]
>>> display_board(board3)
1  C S S E 7030 || || R O C K S
         Guesses              Feedback
```

4.4 `display_key(key: list[str], used_hints: int) -> None`

Displays part of the secret key to the user. The number of positions revealed depends on the given number of used_hints.

A precondition to this function is that the number of used_hints is not negative.

Example:

```
>>> key1 = ['[1]','[2]']
>>> display_key(key1, 0)
Key: ?  ?
>>> key2 = ['[1]','[2]','[1]','[4]']
>>> display_key(key2, 2)
Key: [1]  [2]  ?  ?
>>> key3 = ['I', 'love' , 'Python!']
>>> display_key(key3, 1000)
Key: I  love  Python!
```

4.5 `check_input(command: str) -> bool`

Returns True if `command` is a valid command as given in Table 1, and False otherwise. In the event that `command` is not valid, this function should also display the relevant error message to the user before returning False. Table 2 describes the messages that should be printed when the command is not valid.

Example:

```
>>> check_input('1,2,3,2,1')
True
>>> check_input('H')
True
>>> check_input('1,2,3,2')

Invalid command. Enter 'h' for valid command format or 'q' to quit

False
>>> check_input('1,2,3,2,6')

Invalid number! Available numbers: 1,2,3,4,5

False
```

4.6 `get_command() -> str`

This function should repeatedly prompt the user for a command until they enter a command that is valid according to `check_input`. The function should return the first valid command entered by the user. If the first valid command entered by the user is a guess, this function should format the guess by encasing each number in braces (‘`[`’ and ‘`]`’) before returning it. Otherwise this function should return the command directly.

This function should also result in messages being displayed as described in the specification

for `check_input` whenever the user enters an invalid command.

Example:

```
>>> get_command()
Please enter your guess (h to see valid format): 1,2.4,5,5

Invalid command. Enter 'h' for valid command format or 'q' to quit
Please enter your guess (h to see valid format): 1,2,4,5,G

Invalid number! Available numbers: 1,2,3,4,5

Please enter your guess (h to see valid format): 1,2,4,5,5
'[1],[2],[4],[5],[5]'
>>> get_command()
Please enter your guess (h to see valid format): G

Invalid command. Enter 'h' for valid command format or 'q' to quit
Please enter your guess (h to see valid format): h
'h'
```

4.7 `place_guess(board: list[list[str]], guess: str, row: int) -> None`

Places a given key onto the left hand side of the given board, at the specified row. Any character that is not overridden by a new character should remain unchanged. Note that this function mutates the given board and does *not* return a new board state.

A precondition to this function is that the given guess is a valid length and well formatted. Another precondition to this function is that the given row index starts from 0 and exists in the board.

Example:

```
>>> board = [['[5]','[5]','[4]','[4]','[2]','B','W','W','_','_'], [
'___', '___', '___', '___', '___', '_', '_', '_', '_', '_']]
>>> display_board(board)
1  [5] [5] [4] [4] [2] || || B W W _ _
2  ___ ___ ___ ___ ___ || || _ _ _ _ _
         Guesses              Feedback
>>> place_guess(board, '[1],[1],[1],[1],[1]', 1)
>>> display_board(board)
1  [5] [5] [4] [4] [2] || || B W W _ _
2  [1] [1] [1] [1] [1] || || _ _ _ _ _
         Guesses              Feedback
>>> place_guess(board, '[1],[2],[3],[4],[5]', 0)
>>> display_board(board)
1  [1] [2] [3] [4] [5] || || B W W _ _
2  [1] [1] [1] [1] [1] || || _ _ _ _ _
         Guesses              Feedback
```

4.8 place_feedback(board: list[list[str]], feedback: list[str], row: int) -> None

Places given feedback onto the right hand side of the given board, at the specified row. Any character that is not overridden by a new character should remain unchanged. Note that this function mutates the given board and does *not* return a new board state.

A precondition to this function is that the given feedback is no longer than the length of the secret key. Another precondition to this function is that the given row index starts from 0 exists in the board.
Example:

```
>>> board = [['[5]','[5]','[4]','[4]','[2]','B','W','W','_','_'], [
'___', '___', '___', '___', '___', '_', '_', '_', '_', '_']]
>>> display_board(board)
1  [5] [5] [4] [4] [2] || || B W W _ _
2  ___ ___ ___ ___ ___ || || _ _ _ _ _
         Guesses              Feedback
>>> place_feedback(board,['B','B'], 0)
>>> display_board(board)
1  [5] [5] [4] [4] [2] || || B B W _ _
2  ___ ___ ___ ___ ___ || || _ _ _ _ _
         Guesses              Feedback
>>> place_feedback(board,['B','W','B','W'], 1)
>>> display_board(board)
1  [5] [5] [4] [4] [2] || || B B W _ _
2  ___ ___ ___ ___ ___ || || B W B W _
         Guesses              Feedback
>>> place_feedback(board,[], 1)
>>> display_board(board)
1  [5] [5] [4] [4] [2] || || B B W _ _
2  ___ ___ ___ ___ ___ || || B W B W _
         Guesses              Feedback
>>> place_feedback(board,['B','W','B','W','B'], 0)
>>> display_board(board)
1  [5] [5] [4] [4] [2] || || B W B W B
2  ___ ___ ___ ___ ___ || || B W B W _
         Guesses              Feedback
```

4.9 provide_feedback(key: list[str], guess: str) -> list[str]

Provides feedback on a given guess for a given secret key. Feedback is a list of black and white characters ('B' and 'W' respectively) and should follow the rules given in section 3. See Figure 1 for a labelled example of returned feedback.

A precondition to this function is that the given guess is a comma separated string.

Example:

```
>>> provide_feedback(['[1]','[1]','[1]','[1]','[1]'], '[1],[1],[1],
[1],[1]')
['B', 'B', 'B', 'B', 'B']
```

```
>>> provide_feedback(['[1]','[1]','[1]','[1]','[1]'], '[2],[2],[2],
[2],[2]')
[]
>>> provide_feedback(['[1]','[1]','[2]','[2]','[2]'], '[2],[2],[3],
[2],[1]')
['B', 'W', 'W', 'W']
>>> provide_feedback(['A','B','C'], 'C,D,C')
['B']
>>> provide_feedback(['A','B','C'], 'C,A,C')
['B', 'W']
```

4.10 play_game() -> None

Coordinates gameplay of a single game from start to finish. This function should follow the setup described in section 3, and then follow steps 1 to 7 (inclusive) presented in that same section. The `play_game` function should utilize other functions you have written. In order to make the `play_game` function shorter, you should consider writing extra helper functions.
The output from your `play_game` function (including prompts) must exactly match the expected output. Running the sample tests will give you a good idea of whether your prompts and other outputs are correct. Use samples of gameplay from the `gameplay/` folder provided with this assignment for examples of how the `play_game` function should run.

NOTE: You should make use of the `generate_key` function provided in `support.py` in order to generate the secret key when setting up gameplay. It is imporant that `generate_key` is called ***EXACTLY ONCE*** inside this function. `generate_key` should *never* be called outside `play_game`. If you do not follow these instructions, then the randomly selected code will likely be different to what Gradescope expects, and this can cause you to fail relevant tests.

4.11 main() -> None

The main function should be called when the file is run. The main function enacts a game of Mastermind using the `play_game` function, and then follows step 8 presented in section 3.

The `gameplay/` folder provided with this assignment contains full gameplay examples which should demonstrate how the `main` function should run.

In the provided `a1.py`, the function definition for `main` has already been provided, and the `if __name__ == "__main__":` block will ensure that the code in the `main` function is run when your `a1.py` file is run. Do not call your `main` function outside of this block, and do not call any other function outside this block unless you are calling them from within the body of another function.

5 Assessment and Marking Criteria

This assignment assesses course learning objectives:

1. apply program constructs such as variables, selection, iteration and sub-routines,
2. read and analyse code written by others,
3. read and analyse a design and be able to translate the design into a working program, and

4. apply techniques for testing and debugging.

The assignment will be marked out of 10, with 6 points available for functionality and 4 points available for style.

5.1 Functionality - 6 Marks

Your program's functionality will be marked out of a total of 6 marks. Your assignment will be put through a series of tests and your functionality mark will be proportional to the number of tests you pass. You will be given a *subset* of the functionality tests before the due date for the assignment. You can check if your code passes these functionality tests by uploading it to Gradescope. Note: Functionality tests are automated, so string outputs need to match *exactly* what is expected. You need to perform your *own* testing of your program to make sure that it meets *all* specifications given in the assignment. Only relying on the provided tests is likely to result in your program failing in some cases and you losing some functionality marks. If your final submission does not pass a test (even if this is due to a minor reason such as a missing space) you will not be awarded the marks for that test.

You may receive partial marks within each section for partially working functions, or for implementing only a few functions.

Your program must run in Gradescope, which uses Python 3.12. Partial solutions will be marked but if there are errors in your code that cause the interpreter to fail to execute your program, you will get zero for functionality marks. If there is a part of your code that causes the interpreter to fail, comment out the code so that the remainder can run. Your program must run using the Python 3.12 interpreter. If it runs in another environment (e.g. Python 3.8 or PyCharm) but not in the Python 3.12 interpreter, you will get zero for the functionality mark.

5.2 Code Style - 4 Marks

The style of your assignment will be assessed by a tutor. Style will be marked according to the style rubric provided with the assignment. The style mark will be out of 4.

The key consideration in marking your code style is whether the code is easy to understand. There are several aspects of code style that contribute to how easy it is to understand code. In this assignment, your code style will be assessed against the following criteria.

- Readability
  - Program Structure: Layout of code makes it easy to read and follow its logic. This includes using whitespace to highlight blocks of logic and having **less than 80 characters** in each line of the code.
  - Descriptive Identifier Names: Variable, constant, and function names clearly describe what they represent in the program's logic. Do not use Hungarian Notation for identifiers. In short, this means do not include the identifier's type in its name, rather make the name meaningful (e.g. employee identifier). Any non-trivial fixed value (literal constant) in the code is represented by a descriptive named constant (e.g. `HELP_TEXT`)
- Algorithmic Logic
  - Single Instance of Logic: Blocks of code should not be duplicated in your program. Any code that needs to be used multiple times should be implemented as a function.

  - Variable Scope: Variables should be declared locally in the function in which they are needed. Global variables should not be used.
  - Control Structures: Logic is structured simply and clearly through good use of control structures (e.g. loops and conditional statements).

- Documentation:
  - Comment Clarity: Comments provide meaningful descriptions of the code. They should not repeat what is already obvious by reading the code (e.g. `# Setting variable to 0`). Comments should not be verbose or excessive, as this can make it difficult to follow the code.
  - Informative Docstrings: Every function should have a docstring that summarises its purpose. This includes describing parameters and return values (including type information) so that others can understand how to use the function correctly.
  - Description of Logic: All significant blocks of code should have a comment to explain how the logic works. For a small function, this would usually be the docstring. For long or complex functions, there may be different blocks of code in the function. Each of these should have an in-line comment describing the logic.

5.3 Assignment Submission

You must submit your assignment electronically via Gradescope (`https://gradescope.com/`). You **must** use your UQ email address which is based on your student number (e.g. s4123456@student.uq.edu.au) as your Gradescope submission account.

When you login to Gradescope you may be presented with a list of courses. Select CSSE7030. You will see a list of assignments. Choose **Assignment 1**. You will be prompted to choose a file to upload. The prompt may say that you can upload any files, including zip files. You **must** submit your assignment as a single Python file called `a1.py` (use this name – all lower case), and *nothing* else. Your submission will be automatically run to determine the functionality mark. If you submit a file with a **different name**, the tests will **fail** and you will get **zero** for functionality. Do **not** submit **any** sort of archive file (e.g. zip, rar, 7z, etc.).

Upload an initial version of your assignment *at least* one week before the due date. Do this even if it is just the initial code provided with the assignment. If you are unable access Gradescope, contact the course helpdesk (csse7030@eecs.uq.edu.au) *immediately*. Excuses, such as you were not able to login or were unable to upload a file will not be accepted as reasons for granting an extension.

When you upload your assignment it will run a **subset** of the functionality autograder tests on your submission. It will show you the results of these tests. It is your responsibility to ensure that your uploaded assignment file runs and that it passes the tests you expect it to pass.

Late submissions of the assignment will **not** be marked. Do not wait until the last minute to submit your assignment, as the time to upload it may make it late. Multiple submissions are allowed and encouraged, so ensure that you have submitted an almost complete version of the assignment *well* before the submission deadline of 15:00. Your latest, on time, submission will be marked. Ensure that you submit the correct version of your assignment.

In the event of exceptional personal or medical circumstances that prevent you from handing in the assignment on time, you may submit a request for an extension. See the course profile for

details of how to apply for an extension.

Requests for extensions must be made **before** the submission deadline. The application and supporting documentation (e.g. medical certificate) must be submitted via my.UQ. You must retain the original documentation for a minimum period of six months to provide as verification, should you be requested to do so.

5.4 Plagiarism

This assignment must be your own individual work. By submitting the assignment, you are claiming it is entirely your own work. You **may** discuss general ideas about the solution approach with other students. Describing details of how you implement a function or sharing part of your code with another student is considered to be **collusion** and will be counted as plagiarism. You **may not** copy fragments of code that you find on the Internet to use in your assignment.

Please read the section in the course profile about plagiarism. You are encouraged to complete *both* parts A and B of the academic integrity modules *before* starting this assignment. Submitted assignments will be electronically checked for potential cases of plagiarism.